

Lezione 7: Esercitazione Pratica

Design Pattern, Collezioni Avanzate e Stream

Michele Gentile

26 Maggio 2026

0 Ripasso di Teoria: Design Pattern

🔗 Definizione e Classificazione

Un **Design Pattern** è una soluzione riutilizzabile e collaudata a un problema ricorrente che si presenta durante la progettazione del software. Non si tratta di codice preconfezionato da copiare e incollare, ma di un modello logico da adattare al contesto. I pattern si dividono in tre grandi famiglie principali:

- **Creazionali:** Riguardano i processi di creazione degli oggetti, isolando i dettagli di istanziamento.
- **Strutturali:** Riguardano la composizione di classi e oggetti per formare strutture più grandi e flessibili.
- **Comportamentali:** Si occupano dell'interazione, della ripartizione delle responsabilità e del flusso algoritmico tra gli oggetti.

💡 Tip: Il "Perché"

I pattern esistono per evitare di “reinventare la ruota” e per fornire agli sviluppatori un vocabolario comune. Dire ad un collega “usiamo un Singleton” trasmette immediatamente l'intera struttura logica del codice.

1. I Pattern Creazionali

A. Singleton

🔗 Singleton Pattern

Assicura che una classe abbia una sola istanza in tutta l'applicazione e fornisce un punto di accesso globale a questa istanza. È fondamentale per controllare l'accesso a risorse condivise (es. configurazioni, pool di connessioni a database).

```

public class ClasseSingleton {
    // 1. Campo statico privato che conterra l'unica istanza
    private static ClasseSingleton instance;

    // 2. Costruttore privato per impedire istanziazioni esterne
    private ClasseSingleton() {
        // Inizializzazione della risorsa condivisa
    }

    // 3. Metodo statico pubblico per accedere all'istanza
    public static ClasseSingleton getInstance() {
        if (instance == null) {
            instance = new ClasseSingleton(); // Creazione pigra (
                lazy initialization)
        }
        return instance;
    }
}

```

⚠ Attenzione: Incompletezza del Singleton

Una delle trappole più comuni per gli studenti consiste nel creare correttamente il metodo `getInstance()` dimenticando però di privatizzare il costruttore base. Se il costruttore rimane pubblico, qualsiasi programmatore potrebbe aggirare il pattern creando nuove istanze separate tramite l'operatore `new`.

B. Factory Method & Abstract Factory

🔗 I Pattern di Fabbricazione

- **Factory Method:** Fornisce un'interfaccia per creare oggetti in una superclasse, ma lascia alle sottoclassi la decisione su quale classe esatta istanziare. Disaccoppia il codice principale dalle classi concrete eliminando strutture condizionali rigide.
- **Abstract Factory:** Consente di produrre intere famiglie di oggetti correlati o dipendenti senza specificare le loro classi concrete (es. un negozio di arredamento coordinato in cui ordinando elementi in stile "Moderno" o "Vittoriano" si ottengono prodotti accoppiati coerentemente).

C. Builder

🔗 Builder Pattern

Permette di costruire oggetti complessi passo dopo passo, consentendo allo stesso codice di costruzione di produrre rappresentazioni diverse. Risolve in modo elegante il problema dei **costruttori telescopici** (sovraesposizione di costruttori con combinazioni di parametri opzionali).

```

public class Smartphone {
    private String processore; // Obbligatorio
    private int ram;           // Opzionale
}

```

```

private boolean ha5G;          // Opzionale

private Smartphone(builder builder) {
    this.processore = builder.processore;
    this.ram = builder.ram;
    this.ha5G = builder.ha5G;
}

public static class Builder {
    private String processore;
    private int ram = 4; // Default
    private boolean ha5G = false;

    public Builder(String processore) {
        this.processore = processore;
    }
    public Builder setRam(int ram) {
        this.ram = ram;
        return this;
    }
    public Builder set5G(boolean ha5G) {
        this.ha5G = ha5G;
        return this;
    }

    public Smartphone build() {
        return new Smartphone(this);
    }
}

// Utilizzo client fluida:
Smartphone test = new Smartphone.Builder("A18").setRam(8).set5G(true)
    .build();

```

💡 Tip: Perché i metodi del Builder ritornano un oggetto Builder?

I metodi di configurazione come `setRam()` ritornano `this` (l'istanza corrente del Builder stesso) per consentire il concatenamento dei metodi in cascata (*fluent interface*), rendendo il codice client leggibile ed elegante.

2. I Pattern Strutturali

A. Facade (Facciata)

🔗 Facade Pattern

Fornisce un'interfaccia unificata e semplificata a un sottosistema complesso composto da molte classi interne, nascondendone la complessità di gestione.

Analogia: Il cruscotto o il pulsante di accensione di un'automobile. Per avviare l'auto basta premere un bottone senza dover configurare manualmente l'iniezione della benzina o la scintilla delle candele; la "facciata" si occupa di orchestrare tutto il sottosistema.

B. Decorator

Decorator Pattern

Consente di aggiungere dinamicamente nuove funzionalità e responsabilità a un oggetto esistente avvolendolo in un oggetto wrapper (involucro), senza dover modificare o estendere la classe originale.

Analogia: *Una tazza di caffè base che viene “decorata” aggiungendo progressivamente latte, caramello o panna. Ogni ingrediente modifica il comportamento (prezzo e sapore).*

3. I Pattern Comportamentali

A. Observer

Observer Pattern

Definisce una dipendenza uno-a-molti tra gli oggetti, in modo che quando lo stato di un oggetto (*Subject* o *Observable*) cambia, tutti i suoi dipendenti (*Observers*) vengono notificati e aggiornati automaticamente, eliminando l'inefficiente pratica del *polling*.

Analogia: *L'iscrizione a un canale social o a una newsletter. Non si controlla manualmente ogni ora se il creator ha pubblicato un video; è la piattaforma che invia una notifica automatica quando il contenuto è pronto.*

💡 Tip: Nota Didattica su Observer

Nelle versioni recenti di Java le interfacce e classi standard `Observer` e `Observable` sono state deprecate in favore del paradigma reattivo basato sui **Reactive Streams** (incorporati nella classe `java.util.concurrent.Flow`), che permettono una gestione asincrona robusta del flusso dati.

B. Strategy

Strategy Pattern

Definisce una famiglia di algoritmi, ne incapsula ciascuno e li rende intercambiabili. Permette all'algoritmo applicato di variare dinamicamente a runtime in base alle scelte effettuate dal client.

Analogia: *Il sistema di navigazione di Google Maps. Impostata la destinazione, si può calcolare l'itinerario per andare in auto, a piedi o in bicicletta. L'obiettivo finale non cambia, ma la “strategia” di calcolo varia dinamicamente.*

```
// 1. Definizione della strategia per il volo
interface ComportamentoDiVolo {
    void vola();
}

// 2. Implementazione concreta
class VoloConLeAli implements ComportamentoDiVolo {
    public void vola() {
        System.out.println("Sto volando con le ali!");
    }
}
```

```
// 3. Il contesto che delega tramite composizione
abstract class Anatra {
    ComportamentoDiVolo comportamentoVolo; // Non ereditarietà

    public void eseguiVolo() {
        comportamentoVolo.vola(); // Delega dell'azione
    }
    public void setComportamentoVolo(ComportamentoDiVolo cv) {
        this.comportamentoVolo = cv; // Cambio dinamico a runtime
    }
}
```

⚠ Attenzione: Prediligi la composizione rispetto all'ereditarietà

Se usiamo l'ereditarietà classica per modellare comportamenti variabili, rischiamo di estendere metodi a sottoclassi che non dovrebbero possederli. Ad esempio, aggiungendo il metodo `vola()` nella superclasse `Anatra`, anche un'anatra di plastica giocattolo erediterebbe la capacità di volare. Lo Strategy pattern risolve questo isolando i comportamenti in interfacce funzionali separate usando la composizione.

4. Consolidamento

⚠ Attenzione: Errori frequenti degli studenti

- **L'abuso dei Pattern:** Gli studenti tendono ad applicare i design pattern ovunque, anche in programmi semplicissimi di poche righe, aumentando la complessità ingegneristica del codice senza una reale necessità architetturale.
- **Confusione Ereditarietà vs Composizione:** Molti studenti considerano l'ereditarietà (`extends`) come l'unica via per il riuso del codice. La composizione rende il software decisamente più flessibile a runtime.
- **Incompletezza del Singleton:** Spesso gli studenti creano il metodo `getInstance()` ma dimenticano di privatizzare il costruttore base, permettendo l'uso accidentale di `new ClasseSingleton()`.

💡 Tip: Punti Chiave da Ricordare

- I Design Pattern sono soluzioni architetturali a problemi logici comuni, non librerie da copiare.
- Si dividono in tre categorie principali: Creazionali, Strutturali e Comportamentali.
- Il Singleton garantisce un'unica istanza globale.
- Il Builder elimina l'ingombro dei costruttori telescopici passo dopo passo.
- La Facade semplifica l'accesso a sistemi complessi.
- Lo Strategy incapsula gli algoritmi intercambiabili preferendo la composizione all'ereditarietà.

1 Esercizio 1: Sistema di Configurazione Mecha (Builder e Strategy)

1. Si realizzi una classe **Mecha** caratterizzata da un modello (String, obbligatorio), un'arma principale (String, opzionale, default: "Pugni robotici") e un livello di corazzatura (int, opzionale, default: 100).
2. Per evitare il problema dei costruttori telescopici, si implementi il pattern **Builder** tramite una classe interna statica **Builder** all'interno di *Mecha*, garantendo un'interfaccia fluida (*fluent interface*) per la configurazione dei parametri opzionali. Il costruttore di *Mecha* deve essere rigorosamente **private**.
3. Ogni *Mecha* possiede un comportamento di movimento che può variare dinamicamente a runtime. Si isoli questo comportamento applicando lo **Strategy Pattern**:
 - Si definisca un'interfaccia funzionale **StrategiaMovimento** dotata del metodo `void muovi(String modello)`.
 - Si implementino due strategie concrete: **MovimentoCingolato** (stampa che il mecha avanza cingolando sul terreno) e **MovimentoJetpack** (stampa che il mecha vola bypassando gli ostacoli).
4. La classe *Mecha* deve esporre un metodo `eseguiMovimento()` che delega l'azione alla strategia correntemente impostata e un metodo `setStrategiaMovimento(StrategiaMovimento sm)` per consentire il cambio dinamico della strategia a runtime.

🔗 Obiettivo: Disaccoppiamento Creazionale e Comportamentale

L'obiettivo dell'esercizio è padroneggiare la combinazione dei pattern creazionali (Builder) per la costruzione pulita di oggetti complessi e dei pattern comportamentali (Strategy) per favorire la composizione rispetto all'ereditarietà dogmatica.

💡 Tip: Ritorno del Builder

Non dimenticate di far ritornare **this** a ciascun metodo di configurazione del Builder (es. `setCorazzatura`). In caso contrario, il client non potrà concatenare le chiamate in cascata.

⚠️ Attenzione: Costruttore Visibile

Assicuratevi che il costruttore della classe *Mecha* prenda come parametro un'istanza di *Builder* e sia marcato come **private**. Lasciarlo pubblico permetterebbe la violazione dell'incapsulamento del pattern.

2 Esercizio 2: Piattaforma Analitica di Squadre (Generics, Functional Interfaces e Stream Avanzate)

1. Si progetti una classe generica **RegistroEroi** in cui `T` rappresenta una tipologia di personaggio che espone i metodi `String getNome()`, `String getFazione()` e `int getPotenza()`.
2. La classe *RegistroEroi* deve memorizzare internamente i personaggi utilizzando una collezione di tipo `List`. Si implementi un metodo `aggiungiPersonaggio(T p)` per popolare la struttura.
3. Si implementi un metodo avanzato di analisi statistica chiamato `analizzaFazioni()` che utilizzi le **Stream API** per processare la lista interna. Il metodo deve restituire una mappa di tipo `Map<String, Double>` che associa a ciascuna fazione il **valore medio di potenza** dei personaggi ad essa appartenenti.
4. Si implementi un metodo di filtraggio generico chiamato `filtraEroi(Predicate filtro)` che accetta un'interfaccia funzionale standard e restituisce una `List` contenente solo i personaggi che soddisfano il predicato passato in input.
5. **Fase Avanzata:** Si implementi un metodo `getEroeTopPerFazione()` che restituisce una mappa di tipo `Map<String, Optional>` contenente, per ogni fazione, l'eroe con il livello di potenza massimo, sfruttando i collezionisti di raggruppamento avanzati (`Collectors.groupingBy` combinato con `Collectors.maxBy`).

🔗 Obiettivo: Generics avanzati e Pipeline dichiarativa con Stream

L'esercizio mira a consolidare la programmazione generica limitata (*bounded generics*) e l'utilizzo dei collezionisti complessi delle Stream (`Collectors`), evitando cicli iterativi imperativi tradizionali.

💡 Tip: Prerequisito per la media

Per calcolare la media numerica all'interno di un collettore downstream delle Stream, ricordatevi di mappare l'attributo numerico intero utilizzando `Collectors.averagingInt()`.

⚠️ Attenzione: Gestione degli Optional

Il collettore `Collectors.maxBy` restituisce un valore avvolto in un `Optional`. Assicuratevi che il tipo di ritorno della mappa rifletta correttamente la presenza di `Optional` per evitare errori di compilazione sui tipi generici.